

AUTENTICACIÓN POR FIRMA **EIP-712** PASO A PASO

Un contrato que acepta una firma hecha fuera de la cadena, la verifica y resiste cinco ataques: firma ajena, mensaje alterado, firma vencida, repetición y dominio equivocado.

MODALIDAD	REQUISITO	DURACIÓN	META
En parejas dentro del equipo	laboratorios-evm y keystore (S8)	40 min en clase + Sepolia en casa	16 pruebas verdes + 1 tx en Sepolia

CÓMO LEER ESTA GUÍA

Está escrita para que la hagas **sin ayuda**. Cada paso dice qué escribir, qué deberías ver y cómo saber que funcionó. Si algo no sale como aquí se describe, ve a «**Errores frecuentes**» (sección 10). Las pruebas son el juez: no hay que adivinar si el contrato está bien.

01 Qué vas a construir, y por qué

LEER PRIMERO

Objetivo. Implementar y desplegar un contrato que autentica a una persona por su **firma EIP-712**: el estudiante firma en su billetera, gratis, el mensaje «asistí a la sesión 13»; **otra** cuenta (el computador del docente) envía esa firma y paga el gas; el contrato registra la asistencia solo si la firma es auténtica, vigente y no repetida.

Resultados de aprendizaje. RA4 (desplegar y verificar un contrato con pruebas automatizadas) y RA6 (integrar firmas y eventos entre la interfaz y la cadena).

ARCHIVO	QUÉ ES	QUÉ HACES TÚ
<code>andamiaje/s13/AsistenciaFirmada.sol</code>	El contrato con los TODO 1 a 5.	Lo copias sobre <code>contracts/s13/</code> y lo completas.
<code>test/s13/AsistenciaFirmada.test.js</code>	16 pruebas: 8 de funcionamiento y 8 ataques (★).	No se modifica.
<code>scripts/s13/firmar-asistencia.js</code>	Firma, verifica fuera de la cadena, envía y lee eventos.	Lo corres en local y en Sepolia.
<code>andamiaje/s13/firmar-asistencia.js</code>	El mismo script con los TODO A–F.	Opcional: escribirlo tú.
<code>ignition/modules/s13-AsistenciaFirmada.js</code>	Despliegue reproducible.	Despliegas y verificas en Sepolia.

LA IDEA EN TRES LÍNEAS

1. Quien firma (el estudiante) no gasta nada: firmar no es una transacción. **2.** Quien envía (el docente) paga el gas pero no puede falsificar nada: solo transporta la firma. **3.** El contrato no confía en quien envía: recupera de la firma quién la hizo y lo compara con el campo `estudiante`.

02 Qué se entrega

EVIDENCIA VERIFICABLE

1. Captura de `npx hardhat test test/s13/AsistenciaFirmada.test.js` con **16 passing**.
2. Dirección del contrato **verificado** en Sepolia (enlace a Etherscan, pestaña *Contract* con la marca verde).
3. Hash de la transacción `registrar` en Sepolia y captura de su pestaña *Logs* (o *Events*) con `AsistenciaRegistrada`.
4. La salida completa del script en Sepolia (texto, no foto borrosa).
5. Respuestas a las preguntas de reflexión (sección 11), con los valores **de tu** despliegue.

Todo en un solo documento por pareja, por el canal del curso. Política del curso: sin evidencia en cadena, el laboratorio no se califica.

03 Preparación desde cero

10 MINUTOS

Si ya hiciste los laboratorios de las sesiones 8 a 12 en este mismo computador, solo actualiza el repositorio (paso 3.3). Si no, haz todo.

3.1 • Comprobar versiones

```
node --version          # debe decir v22.x (Node 22 LTS)
git --version           # cualquier versión reciente
npx hardhat --version   # dentro de laboratorios-evm: 3.16.0
```

Cómo sé que funcionó: los tres comandos responden un número de versión. Si `node` no se reconoce, instala Node.js 22 LTS desde nodejs.org y abre una terminal nueva.

3.2 • Obtener el repositorio (solo la primera vez)

```
git clone <URL del repositorio del curso>
cd laboratorios-evm
npm install                # una sola vez; tarda unos minutos
```

3.3 • Actualizar (si ya lo tenías)

```
cd laboratorios-evm
git pull
npx hardhat test test/s13/AsistenciaFirmada.test.js    # debe COMPILAR; cuántas pasan se ve en el
paso 4
```

Cómo sé que funcionó: existen las carpetas `contracts/s13`, `test/s13`, `scripts/s13` y `andamiaje/s13`. La primera corrida de pruebas compila sin errores. El repositorio del curso ya trae el contrato con huecos; si alguna vez ves `16 passing` antes de escribir nada, tienes una versión completa y el paso 4 la reemplaza por el andamiaje.

3.4 • El keystore (para Sepolia, sección 8)

```
npx hardhat keystore list          # deben aparecer los tres nombres de abajo
npx hardhat keystore set SEPOLIA_RPC_URL    # si falta alguno
npx hardhat keystore set SEPOLIA_PRIVATE_KEY
npx hardhat keystore set ETHERSCAN_API_KEY
```

REGLAS QUE NO ADMITEN EXCEPCIÓN

La clave privada es la de la **billettera del curso**, sin fondos reales. Claves y URLs de RPC van **solo** en el keystore: nunca en un archivo, nunca en un chat, nunca en una captura. La frase semilla no se comparte con nadie, tampoco con el docente. Y no firmes nada que no entiendas: en este laboratorio sabes exactamente qué firmas, campo por campo.

04 Paso 1 · Copiar el andamiaje y ver el rojo

5 MINUTOS

```
copy andamiaje\s13\AsistenciaFirmada.sol contracts\s13\      (Windows)
cp andamiaje/s13/AsistenciaFirmada.sol contracts/s13/        (macOS / Linux)
npx hardhat test test/s13/AsistenciaFirmada.test.js
```

Salida esperada (resumida):

```
Warning: Function state mutability can be restricted to pure      ← normal en el andamiaje
S13 · AsistenciaFirmada · el dominio
  √ expone su dominio EIP-712 (nombre, versión, red y dirección)
  1) calcula el mismo resumen que ethers (TypedDataEncoder)
...
3 passing
13 failing
```

Cómo sé que funcionó: compila, y salen exactamente **3 verdes y 13 rojas**. Las tres verdes no dependen de tu código (el dominio lo arma OpenZeppelin; firmar no cuesta gas; quien envía es quien paga). Las 13 rojas son el mapa de lo que falta.

SI QUIERES VOLVER A EMPEZAR

Vuelve a copiar el andamiaje encima: `andamiaje/s13/` nunca se modifica. Para deshacer cualquier cambio y volver a lo que trae el repositorio: `git checkout contracts/s13/AsistenciaFirmada.sol`.

05 Paso 2 · TODO 1: el resumen que se firma

PUNTO DE CONTROL A LOS 20 MIN

Una billetera no firma «el mensaje»: firma un **resumen** de 32 bytes calculado así (EIP-712):

```
hashStruct = keccak256( abi.encode( TYPEHASH, estudiante, sesion, nonce, vence ) )
resumen    = keccak256( 0x19 0x01 || separadorDeDominio || hashStruct )
```

La segunda línea la hace OpenZeppelin con `_hashTypedDataV4`, porque tu contrato hereda `EIP712("AsistenciaUSB", "1")`. El separador de dominio mezcla nombre, versión, `chainId` y la dirección del contrato. Tú escribes la primera.

1

FUNCIÓN · hashAsistencia(estudiante, sesion, nonce, vence)

Devolver el resumen EIP-712

Qué escribir: borra las dos líneas de relleno y escribe:

```
bytes32 hashStruct = keccak256(abi.encode(ASISTENCIA_TYPEHASH, estudiante, sesion,
nonce, vence));
return _hashTypedDataV4(hashStruct);
```

Tres detalles que rompen todo si se hacen mal:

- `abi.encode`, no `abi.encodePacked`: EIP-712 exige que cada campo ocupe 32 bytes.
- Los campos en el MISMO orden que en el tipo: `estudiante`, `sesion`, `nonce`, `vence`.
- `ASISTENCIA_TYPEHASH` ya está escrito: no le cambies ni un espacio.

Cómo lo compruebas: `npx hardhat test test/sl3/AsistenciaFirmada.test.js` → ahora pasan también «calcula el mismo resumen que ethers» y «fuera de la cadena, `verifyTypedData` recupera a quien firmó». Esa primera prueba compara tu hash con el que calcula `ethers.TypedDataEncoder.hash`: si coinciden, tu Solidity y la billetera hablan el mismo idioma.

06 Paso 3 · TODO 2 a 5: registrar

EL NÚCLEO

La función `registrar(estudiante, sesion, nonce, vence, firma)` la puede llamar cualquiera. Su trabajo es decidir si la firma autoriza lo que dice. Escríbela en este orden:

2

TODO 2 · VENCIMIENTO

¿Sigue vigente?

```
if (block.timestamp > vence) revert FirmaVencida(vence, block.timestamp);
```

Por qué: sin fecha límite, una firma robada hoy serviría dentro de un año. Se compara con la hora del **bloque**, no la del computador.

3

TODO 3 · AUTENTICIDAD

¿La firmó el estudiante, para este contrato y esta red?

```
address recuperado = ECDSA.recover(hashAsistencia(estudiante, sesion, nonce, vence),
firma);
if (recuperado != estudiante) revert FirmaInvalida(recuperado, estudiante);
```

Por qué funciona: es la recuperación de clave pública de la Sesión 3. Si cualquier bit del mensaje o del dominio difiere de lo que se firmó, el resumen cambia y `recover` devuelve OTRA dirección (no falla: devuelve una dirección que no es la del estudiante). Por eso comparamos.

4

TODO 4 · ANTI-REPETICIÓN

¿Es el nonce que toca? Consumirlo

```
_useCheckedNonce(estudiante, nonce);
```

Qué hace: viene de Nonces de OpenZeppelin 5. Si nonce no es el siguiente del estudiante, revierte con InvalidAccountNonce(estudiante, nonceActual) ; si lo es, lo incrementa. La segunda vez que alguien envíe la misma firma, el nonce ya no coincide.

5

TODO 5 · EFECTO Y EVENTO

Registrar y dejar rastro

```
asistio[sesion][estudiante] = true;
emit AsistenciaRegistrada(estudiante, sesion, msg.sender, nonce);
```

Por qué el evento: es lo que después se lee con queryFilter o se indexa (bloque A de la clase). msg.sender es quien envió, no quien firmó: el evento guarda los dos.

Cómo sé que terminé: `npx hardhat test test/s13/AsistenciaFirmada.test.js` muestra **16 passing**. Revisa en particular las ocho con ★: son los ataques de la tabla A.9 de la clase.

```
S13 · AsistenciaFirmada · ataques que debe resistir
✓ ★ firma de otro: Beto no puede marcar la asistencia de Ana
✓ ★ mensaje alterado: cambiar la sesión después de firmar invalida la firma
✓ ★ firma vencida: pasada la fecha `vence`, se rechaza
✓ ★ repetición: la misma firma no se puede usar dos veces (nonce consumido)
✓ ★ nonce adelantado: una firma con nonce 5 no sirve si el siguiente es 0
✓ ★ dominio de otra cadena: una firma hecha para Sepolia no sirve en esta red
✓ ★ dominio de otro contrato: una firma para otra copia del contrato no sirve aquí
✓ ★ una firma mal formada se rechaza con el error de ECDSA
16 passing
```

Opcional, y recomendado: `npx hardhat test test/s13/AsistenciaFirmada.test.js --coverage`. La solución de referencia da 100 % de líneas en `AsistenciaFirmada.sol`.

07 Paso 4 · El script en la red local

FIRMAR ≠ TRANSACCIONAR

```
npx hardhat run scripts/s13/firmar-asistencia.js
```

El script despliega un contrato nuevo en la red local, usa la cuenta 1 como estudiante (solo firma) y la cuenta 0 como docente (envía y paga). **Salida real** de una corrida (la firma y los hashes cambian con la hora, porque `vence` depende del bloque):

```

1 · DOMINIO
  nombre / versión : AsistenciaUSB / 1
  chainId          : 31337
  contrato         : 0x5FbDB2315678afecb367f032d93F642f64180aa3
2 · MENSAJE QUE SE FIRMA
  estudiante       : 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
  sesión / nonce   : 13 / 0
3 · FIRMA (65 bytes = r || s || v)
  0x07a7d7f09c20a508...c43673eb1b
  transacciones del estudiante antes/después de firmar: 0 / 0 ← firmar no es transaccionar
4 · VERIFICACIÓN FUERA DE LA CADENA
  digest EIP-712   : 0x32c9f403bd790a136c211e4462f7bdd20db24d597a7beed337f7ef5546d455f1
  digest (contrato): 0x32c9f403bd790a136c211e4462f7bdd20db24d597a7beed337f7ef5546d455f1
  recuperado       : 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 ✓ coincide
5 · ENVÍO EN CADENA
  enviada por      : 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266 (el docente paga el gas)
  bloque / gas     : 2 / 75330
  ¿asistió?       : true
6 · EVENTOS LEÍDOS CON queryFilter (bloques 0 a latest)
  bloque 2 · sesión 13 · nonce 0 · enviada por 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266

```

Cómo sé que funcionó: los dos *digest* son idénticos, aparece «✓ coincide», el contador de transacciones del estudiante es igual antes y después de firmar, y el evento aparece en el paso 6.

OPCIONAL · ESCRIBIR EL SCRIPT TÚ MISMO

Copia `andamiaje/s13/firmar-asistencia.js` sobre `scripts/s13/firmar-asistencia.js` y completa los TODO A–F: el dominio (léelo con `eip712Domain()`), los tipos, `signTypedData`, `verifyTypedData`, el envío y `queryFilter`. Termina cuando imprime «✓ coincide» y un hash de transacción. Restaura el original con `git checkout scripts/s13/firmar-asistencia.js`.

08 Paso 5 · Sepolia: la evidencia en cadena

EN CLASE O EN CASA

5.1 · Desplegar y verificar.

```
npx hardhat ignition deploy ignition/modules/s13-AsistenciaFirmada.js --network sepolia --verify
```

Salida esperada: `AsistenciaFirmadaModulo#AsistenciaFirmada - 0x...` y un mensaje de verificación exitosa con el enlace a `sepolia.etherscan.io`. Si el keystore tiene contraseña, Hardhat la pide. Copia la dirección.

5.2 · Firmar y enviar sobre TU contrato. En Sepolia solo hay una cuenta configurada: la misma firma y envía (el script lo dice).

```

$env:CONTRATO="0x...la dirección de 5.1..." (PowerShell)
export CONTRATO=0x...la dirección de 5.1... (macOS / Linux)
npx hardhat run scripts/s13/firmar-asistencia.js --network sepolia

```

Tarda unos 15 a 30 segundos: espera a que se mine el bloque. La dirección de un contrato **no** es un secreto; por eso puede ir en una variable de entorno. La clave, jamás.

5.3 · Comprobar en Etherscan. Abre `https://sepolia.etherscan.io/tx/<hash>` : estado *Success*, *From* = tu cuenta, *To* = tu contrato. En la pestaña *Logs* verás `AsistenciaRegistrada` con cuatro *topics*. En la página del contrato, pestaña *Contract*, la marca verde de verificado; en *Read Contract* puedes consultar `asistio(13, tuDireccion) → true`.

Cómo sé que terminé: tengo dirección verificada, hash de `registrar` con estado *Success* y la captura del log.

09 Puntos de control

MINUTO	DEBERÍAS TENER	SI NO LO TIENES
5	3 passing / 13 failing con el andamiaje.	Revisa la sección 3 y la carpeta en la que estás.
20	TODO 1 listo: pasa «calcula el mismo resumen que ethers».	El docente proyecta la solución del TODO 1 (lámina B.3).
35	16 passing.	Compara tu <code>registrar</code> con la lámina B.3.
40	Script local con «✓ coincide».	Sepolia queda para casa; lo local es obligatorio en clase.

10 Errores frecuentes

CASI TODO SE RESUELVE AQUÍ

SÍNTOMA	CAUSA Y SOLUCIÓN
«calcula el mismo resumen» falla con hashes distintos	Usaste <code>abi.encodePacked</code> , cambiaste el orden de los campos o tocaste <code>ASISTENCIA_TYPEHASH</code> . Deben ser idénticos al tipo que firma ethers.
Todas las ★ de dominio pasan pero «firma válida» falla con FirmaInvalida	Pasaste los argumentos a <code>hashAsistencia</code> en otro orden dentro de <code>registrar</code> .
«repetición» no revierte	Falta el TODO 4, o usaste <code>nonces (estudiante)</code> sin consumirlo. <code>_useCheckedNonce</code> compara Y incrementa.
La repetición revierte con FirmaInvalida en vez de InvalidAccountNonce	Recalculas el resumen con <code>nonces (estudiante)</code> (que ya subió) en vez del <code>nonce</code> recibido. Usa el parámetro.
«firma mal formada» espera ECDSAInvalidSignatureLength y obtiene otro error	No usaste <code>ECDSA.recover</code> de OpenZeppelin (o usaste <code>ecrecover</code> directo, que no valida longitud).
HHE... / «Cannot find module» al correr el script	No estás dentro de <code>laboratorios-evm</code> , o falta <code>npm install</code> .
En Sepolia: «insufficient funds»	La billetera del curso no tiene ETH de Sepolia. Pídelo al docente (billetera institucional).
En Sepolia: FirmaVencida	La transacción tardó más de 10 minutos en minarse (red congestionada). Vuelve a correr el script: firma con un <code>vence</code> nuevo.
En Sepolia: la verificación de Etherscan falla	Falta <code>ETHERSCAN_API_KEY</code> en el keystore, o Etherscan todavía no indexó el contrato. Espera un minuto y verifica sin redespargar: <code>npx hardhat ignition verify chain-11155111</code> .
queryFilter no devuelve nada en Sepolia	Rango de bloques demasiado grande para el RPC. El script busca desde el bloque del recibo menos 10; si lo modificaste, vuelve a acotar el rango.

11 Preguntas de reflexión

CON TUS PROPIOS VALORES

Se responden mirando **tu** despliegue y **tu** salida. Una respuesta sin el valor concreto pedido no cuenta.

1. **Gas.** Copia el `gasUsed` de tu transacción `registrar` en Sepolia (Etherscan, *Gas Usage by Txn*) y compáralo con los 75 330 de la red local. ¿Coinciden exactamente? Si no, ¿qué parte de la transacción cambia entre una firma y otra y cuesta gas distinto? (Pista: Sesión 5, costo del calldata por byte cero y no cero.)
2. **Topics.** En la pestaña *Logs* de tu transacción, copia los cuatro *topics*. ¿Cuál es el `topic0`? Compruébalo con `ethers.id("AsistenciaRegistrada(address,uint256,address,uint256)")` y di qué contienen los topics 1, 2 y 3. ¿Por qué el `nonce` no aparece como topic?

- Mensaje alterado.** En una consola (`npx hardhat console --network sepolia` o un script), toma tu firma de Sepolia y llama `ethers.verifyTypedData` con el mismo dominio pero `sesion: 14` . Copia la dirección que devuelve. ¿Es la tuya? ¿Por qué `recover` no «falla» sino que devuelve otra dirección?
- El dominio en cadena.** En *Read Contract* de Etherscan, consulta `eip712Domain` . Copia `chainId` y `verifyingContract` . ¿Qué ataque de la tabla A.9 cierra cada uno?
- Orden de las comprobaciones.** Si en `registrar` intercambias los TODO 3 y 4 (primero consumir el nonce, después verificar la firma), ¿se rompe la seguridad? Justifica con lo que deshace un `revert` , y di qué recomienda el patrón checks-effects-interactions.
- Diseño.** En este contrato cualquiera puede enviar la firma de Ana. Describe un escenario de su proyecto donde eso sea una ventaja (gas patrocinado) y otro donde sería un problema, y qué campo añadirían al tipo para evitarlo.

12 Lista de chequeo de entrega

✓	ÍTEM
☐	Captura con <code>16 passing</code> de <code>test/s13/AsistenciaFirmada.test.js</code> , con tu versión del contrato (no la de referencia).
☐	Dirección del contrato en Sepolia con enlace y marca de verificado.
☐	Hash de la transacción <code>registrar</code> con estado <i>Success</i> y captura de sus <i>Logs</i> .
☐	Salida completa del script en Sepolia, como texto.
☐	Las seis respuestas de reflexión con los valores de tu despliegue.
☐	Nada de lo entregado contiene una clave privada, una frase semilla ni una URL de RPC con API key.

13 Rúbrica

ESCALA DE 0 A 5,0

CRITERIO	PESO	EXCELENTE (5,0)	ACEPTABLE (3,5)	INSUFICIENTE (<3,0)
Contrato y pruebas	40 %	16 passing con el código propio; sin cambios a las pruebas.	13 a 15 pruebas verdes.	Menos de 13, o se modificaron las pruebas.
Evidencia en Sepolia	30 %	Contrato verificado, <code>registrar</code> exitoso, log capturado.	Desplegado y usado, pero sin verificar.	Sin evidencia en cadena (no se califica el laboratorio).
Reflexión	30 %	Las seis respuestas, con valores propios y razonamiento correcto.	Respuestas correctas pero sin los valores del despliegue propio.	Respuestas genéricas, sin mirar el artefacto.

14 Referencias

- EIP-712: *Typed structured data hashing and signing*. <https://eips.ethereum.org/EIPS/eip-712>
- ERC-4361: *Sign-In with Ethereum* (firma texto con ERC-191). <https://eips.ethereum.org/EIPS/eip-4361>
- ERC-2612: *Permit*, la misma técnica aplicada a tokens. <https://eips.ethereum.org/EIPS/eip-2612>
- OpenZeppelin Contracts 5.x · EIP712, ECDSA y Nonces.
<https://docs.openzeppelin.com/contracts/5.x/api/utils/cryptography>
- ethers v6 · `Signer.signTypedData` y `verifyTypedData`.
<https://docs.ethers.org/v6/api/providers/#Signer-signTypedData>